



ÉPICO: MÓDULO MARKETPLACE



OBJETIVO GENERAL

Crear un marketplace interno donde los usuarios pueden publicar y vender gallos, con control de límites por plan de suscripción y gestión completa de publicaciones.



ANÁLISIS DE BASE DE DATOS



Nueva Tabla: marketplace_publicaciones

```
CREATE TABLE public.marketplace_publicaciones (  
    id serial4 NOT NULL,  
    user_id int4 NOT NULL,  
    titulo varchar(255) NOT NULL,  
    descripcion text NULL,  
    precio numeric(10, 2) NOT NULL,  
    galpop_nombre varchar(100) NOT NULL,  
    vendedor_nombre varchar(100) NOT NULL,  
    vendedor_telefono varchar(20) NOT NULL,  
    vendedor_email varchar(100) NOT NULL,  
    foto_1_url text NULL,  
    foto_2_url text NULL,  
    foto_3_url text NULL,  
    foto_4_url text NULL,  
    estado varchar(50) DEFAULT 'venta'::character varying NOT NULL, -- 'venta', 'vendido', 'pau:  
    ubicacion varchar(100) NULL,  
    fecha_nacimiento_gallo date NULL,  
    raza varchar(100) NULL,  
    peso_gramos int4 NULL,  
    created_at timestamp DEFAULT CURRENT_TIMESTAMP NULL,  
    updated_at timestamp DEFAULT CURRENT_TIMESTAMP NULL,  
    CONSTRAINT marketplace_publicaciones_pkey PRIMARY KEY (id),  
    CONSTRAINT marketplace_publicaciones_user_id_fkey FOREIGN KEY (user_id) REFERENCES public.u:  
    CONSTRAINT check_estado_valido CHECK (estado IN ('venta', 'vendido', 'pausado')),  
    CONSTRAINT check_precio_positivo CHECK (precio > 0)  
);  
  
-- Índices para optimización  
CREATE INDEX idx_marketplace_user ON public.marketplace_publicaciones USING btree (user_id);  
CREATE INDEX idx_marketplace_estado ON public.marketplace_publicaciones USING btree (estado);  
CREATE INDEX idx_marketplace_precio ON public.marketplace_publicaciones USING btree (precio);  
CREATE INDEX idx_marketplace_fecha ON public.marketplace_publicaciones USING btree (created_at);  
CREATE INDEX idx_marketplace_ubicacion ON public.marketplace_publicaciones USING btree (ubicacion);
```



Modificación Tabla: planes_catalogo

-- Agregar nueva columna para límites de marketplace

```
ALTER TABLE public.planes_catalogo
```

```
ADD COLUMN marketplace_publicaciones_max int4 DEFAULT 0;
```

-- Valores por defecto por plan

```
UPDATE public.planes_catalogo SET marketplace_publicaciones_max = 0 WHERE codigo = 'gratis';
```

```
UPDATE public.planes_catalogo SET marketplace_publicaciones_max = 3 WHERE codigo = 'basico';
```

```
UPDATE public.planes_catalogo SET marketplace_publicaciones_max = 5 WHERE codigo = 'premium';
```

```
UPDATE public.planes_catalogo SET marketplace_publicaciones_max = 10 WHERE codigo = 'profesional';
```



FLUJO COMPLETO DEL SISTEMA



FLUJO USUARIO



Pestaña 1: EXPLORAR MARKETPLACE

1. Usuario ve todas las publicaciones activas

- Lista de gallos en venta de todos los usuarios
- Información visible: foto principal, precio, galpón, vendedor
- Filtros: precio, ubicación, raza, fecha

2. Usuario ve detalle de publicación

- Galería de hasta 4 fotos
- Información completa del gallo
- Datos de contacto del vendedor
- Botón "Contactar Vendedor" (WhatsApp/Llamada)



Pestaña 2: MIS PUBLICACIONES

1. Usuario ve sus publicaciones

- Lista de sus gallos publicados
- Estados: En venta, Vendido, Pausado
- Estadísticas: vistas, contactos recibidos

2. Usuario crea nueva publicación

- Verifica límite según su plan
- Formulario auto-rellena datos del perfil

- Sube hasta 4 fotos del gallo
- Define precio y detalles

3. **Usuario gestiona publicaciones**

- Editar información y fotos
- Cambiar estado (venta/vendido/pausado)
- Eliminar publicación
- Ver estadísticas



ARQUITECTURA DE DESARROLLO



FRONTEND (Flutter)



Nuevos Archivos a Crear:

```
lib/
├── features/
│   ├── marketplace/
│   │   ├── models/
│   │   │   ├── marketplace_models.dart          # Nuevo
│   │   │   └── publicacion_models.dart          # Nuevo
│   │   ├── screens/
│   │   │   ├── marketplace_screen.dart          # Nuevo (pantalla principal con tabs)
│   │   │   ├── explorar_tab.dart                # Nuevo (pestaña 1)
│   │   │   ├── mis_publicaciones_tab.dart        # Nuevo (pestaña 2)
│   │   │   ├── detalle_publicacion_screen.dart  # Nuevo
│   │   │   ├── crear_publicacion_screen.dart     # Nuevo
│   │   │   └── editar_publicacion_screen.dart    # Nuevo
│   │   ├── widgets/
│   │   │   ├── publicacion_card.dart            # Nuevo
│   │   │   ├── galeria_fotos_widget.dart        # Nuevo
│   │   │   ├── filtros_marketplace_widget.dart  # Nuevo
│   │   │   └── subir_fotos_widget.dart          # Nuevo
│   │   └── services/
│   │       └── marketplace_service.dart          # Nuevo
│   └── shared/
│       ├── widgets/
│       │   └── contacto_vendedor_widget.dart    # Nuevo
│       └── utils/
│           └── url_launcher_helper.dart          # Nuevo (WhatsApp, llamadas)
```

Modificaciones en Home:

`lib/features/home/screens/home_screen.dart`

- Reordenar módulos según nueva estructura:

1.  Transmisiones
2.  Marketplace # NUEVO
3.  Pedigrí
4.  Peleas
5.  Topes
6.  Vacunas
7.  Reportes



BACKEND (FastAPI)



Nuevos Endpoints:

```
# marketplace.py
@router.get("/marketplace/publicaciones")
async def listar_publicaciones_activas(
    skip: int = 0,
    limit: int = 20,
    precio_min: float = None,
    precio_max: float = None,
    ubicacion: str = None,
    raza: str = None
)

@router.get("/marketplace/publicaciones/{publicacion_id}")
async def obtener_detalle_publicacion(publicacion_id: int)

@router.get("/marketplace/mis-publicaciones")
async def listar_mis_publicaciones(current_user: User = Depends(get_current_user))

@router.post("/marketplace/publicaciones")
async def crear_publicacion(
    publicacion: PublicacionCreate,
    current_user: User = Depends(get_current_user)
)

@router.put("/marketplace/publicaciones/{publicacion_id}")
async def actualizar_publicacion(
    publicacion_id: int,
    publicacion: PublicacionUpdate,
    current_user: User = Depends(get_current_user)
)

@router.delete("/marketplace/publicaciones/{publicacion_id}")
async def eliminar_publicacion(
    publicacion_id: int,
    current_user: User = Depends(get_current_user)
)

@router.post("/marketplace/publicaciones/{publicacion_id}/fotos")
async def subir_foto_publicacion(
    publicacion_id: int,
```

```

    foto_numero: int, # 1, 2, 3, o 4
    file: UploadFile,
    current_user: User = Depends(get_current_user)
)

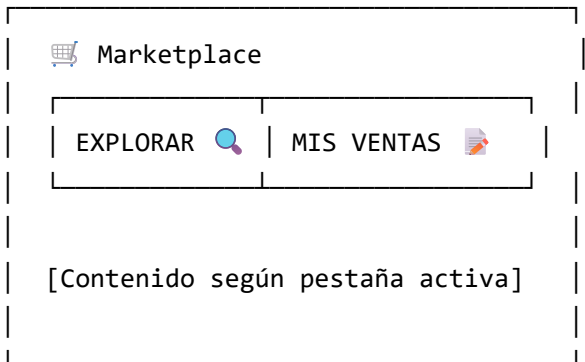
@router.patch("/marketplace/publicaciones/{publicacion_id}/estado")
async def cambiar_estado_publicacion(
    publicacion_id: int,
    nuevo_estado: str, # 'venta', 'vendido', 'pausado'
    current_user: User = Depends(get_current_user)
)

# Verificación de límites
@router.get("/marketplace/limites")
async def verificar_limites_marketplace(current_user: User = Depends(get_current_user))

```

DISEÑO DE INTERFAZ

Pantalla Principal: MarketplaceScreen



Pestaña 1: Explorar

 Explorar Marketplace	
 Filtros: [Precio] [Ubicación]	
[FOTO]	Gallo Champion
	 S/ 1,500
	 Galpón El Dorado
	 Juan Pérez
	 Lima, Perú
[FOTO]	Gallo Furia
	 S/ 800
	 Criadero Norte
	 María García
	 Trujillo, Perú



Pestaña 2: Mis Publicaciones



Mis Publicaciones



Usadas: 2/3 (Plan Básico)



Nueva Publicación

[FOTO]



Mi Gallo Campeón



S/ 1,200



En Venta



45 vistas



[Editar]



[Del]

[FOTO]



Gallo Vendido



S/ 900



Vendido



67 vistas



[Ver]

Detalle de Publicación

 Gallo Champion



[FOTO 1] [FOTO 2] [FOTO 3] [4]

Galería de Fotos

 Precio: S/ 1,500

 Galpón: El Dorado

 Nacimiento: 15/03/2023

 Peso: 2,100 gramos

 Raza: Shamo

 Vendedor: Juan Pérez

 Teléfono: +51 987 654 321

 Email: juan@email.com

 Ubicación: Lima, Perú

 Contactar por WhatsApp

 Llamar Ahora

+ Crear Nueva Publicación

+ Nueva Publicación



Fotos del Gallo (máx. 4)

[+]	[+]	[+]	[+]
			

Información Básica

Título: [_____]
Precio: S/ [_____]
Descripción:
[_____]
[_____]
[_____]

Datos del Gallo

Fecha Nacimiento: [DD/MM/AAAA]
Peso (gramos): [_____]
Raza: [_____]
Ubicación: [_____]

Datos Vendedor (Auto-rellenado)

Galpón: Mi Galpón
Nombre: Juan Pérez
Teléfono: +51 987 654 321
Email: juan@email.com

Publicar Gallo



CONTROL DE LÍMITES POR PLAN



Límites por Suscripción:

-  **Gratuito:** 0 publicaciones (sin acceso)
-  **Básico:** 3 publicaciones activas máximo
-  **Premium:** 5 publicaciones activas máximo
-  **Profesional:** 10 publicaciones activas máximo



Validaciones:

1. **Al crear publicación:** Verificar límite antes de permitir creación
2. **Al cambiar plan:** Si baja de plan y excede límite, pausar publicaciones excedentes
3. **Estados contabilizados:** Solo 'venta' y 'pausado' cuentan para el límite
4. **Estado 'vendido':** No cuenta para el límite (historial)



HISTORIAS DE USUARIO



Como Usuario:

1. **US-01: Explorar Marketplace**
 - Como usuario, quiero ver todos los gallos en venta
 - Para encontrar gallos que me interesen comprar
2. **US-02: Filtrar Publicaciones**
 - Como usuario, quiero filtrar por precio, ubicación y raza
 - Para encontrar gallos específicos que busco
3. **US-03: Ver Detalle de Gallo**
 - Como usuario, quiero ver información completa y fotos
 - Para evaluar si me interesa el gallo
4. **US-04: Contactar Vendedor**
 - Como usuario, quiero contactar al vendedor fácilmente

- Para negociar la compra del gallo

5. **US-05: Publicar Mi Gallo**

- Como usuario, quiero publicar mis gallos en venta
- Para venderlos a otros criadores

6. **US-06: Gestionar Mis Publicaciones**

- Como usuario, quiero editar, pausar o eliminar mis publicaciones
- Para mantener mi inventario actualizado

7. **US-07: Ver Límites de Plan**

- Como usuario, quiero saber cuántas publicaciones puedo hacer
- Para gestionar mi suscripción adecuadamente



PLAN DE IMPLEMENTACIÓN



Sprint 1: Base de Datos y Backend

- ☐ Crear tabla `marketplace_publicaciones`
- ☐ Modificar tabla `planes_catalogo`
- ☐ Implementar endpoints básicos CRUD
- ☐ Sistema de límites por plan
- ☐ Pruebas de API



Sprint 2: Frontend - Explorar

- ☐ Pantalla principal con tabs
- ☐ Pestaña "Explorar" marketplace
- ☐ Lista de publicaciones con filtros
- ☐ Detalle de publicación
- ☐ Widget de contacto (WhatsApp/llamada)



Sprint 3: Frontend - Mis Publicaciones

- ☐ Pestaña "Mis Publicaciones"
- ☐ Crear nueva publicación
- ☐ Subir hasta 4 fotos
- ☐ Editar publicaciones existentes
- ☐ Cambiar estados (venta/vendido/pausado)



Sprint 4: Integración y Pulido

- ☐ Integrar con sistema de planes
- ☐ Validaciones de límites en tiempo real
- ☐ Reordenar módulos en home
- ☐ Mejoras de UX/UI
- ☐ Pruebas de usuario



CONSIDERACIONES TÉCNICAS



Gestión de Imágenes

- **Almacenamiento:** Cloudinary
- **Límite por foto:** 5MB máximo
- **Formatos:** JPG, PNG, WebP
- **Optimización:** Compresión automática
- **Miniaturas:** Generación automática para lista



Contacto con Vendedores

- **WhatsApp:** url_launcher con link directo
- **Llamadas:** url_launcher con tel:
- **Email:** url_launcher con mailto:
- **Validación:** Verificar aplicaciones instaladas



Búsqueda y Filtros

- **Filtros básicos:** Precio, ubicación, raza
- **Ordenamiento:** Precio, fecha, relevancia
- **Búsqueda:** Por título y descripción
- **Paginación:** 20 publicaciones por página



Seguridad

- **Autenticación:** JWT token obligatorio
- **Autorización:** Solo owner puede editar/eliminar
- **Validación:** Límites por plan en backend

- **Moderación:** Flag para reportar publicaciones



MÉTRICAS DE ÉXITO

1. Adopción

- Número de usuarios que publican
- Publicaciones creadas por mes
- Tiempo promedio hasta primera publicación

2. Engagement

- Publicaciones vistas por día
- Contactos generados por publicación
- Tasa de conversión vista → contacto

3. Monetización

- Upgrades de plan motivados por límites
- Retención de usuarios con publicaciones activas

4. Calidad

- Publicaciones con 4 fotos vs menos fotos
- Tiempo promedio de publicación activa
- Tasa de publicaciones marcadas como vendidas



RESULTADO ESPERADO

Al finalizar este épico, tendremos:

- ✓ **Marketplace funcional** con exploración y gestión
- ✓ **Sistema de límites** integrado con planes de suscripción
- ✓ **Herramientas de contacto** para facilitar ventas
- ✓ **Gestión completa** de publicaciones por usuario
- ✓ **Interfaz intuitiva** para comprar y vender gallos
- ✓ **Nuevo módulo principal** en el home de la app
- ✓ **Incentivo adicional** para upgrades de suscripción